Министерство образования Республики Беларусь

Учреждение образования «Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в ИС»

Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:

Студент 3 курса

Группы ИИ-26 (1)

Згера Е. А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:
   - Заменить пороговую функцию активации на сигмоидную.
   - Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE). ▯

Реализовать вычисление градиента ŷ-exi для обновления весов. 2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом: ▯ На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
   - Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE. ▯ Сохранять значения Esp для построения графика.
4. Провести сравнительный анализ:
   - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
   - MSE + фиксированный шаг (данные из ЛР №1);
   - MSE + адаптивный шаг (данные из ЛР №2); ▯ BCE + фиксированный шаг; ▯ BCE + адаптивный шаг.
   - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).
5. Визуализировать результаты:
   - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
   - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
   - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
   - Пользователь вводит координаты x1,x2.
   - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
   - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE.

Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Вариант 5:

5.

x_1	x_2	e
2	6	0
-2	6	1
2	-6	0
-2	-6	0

Код программы:

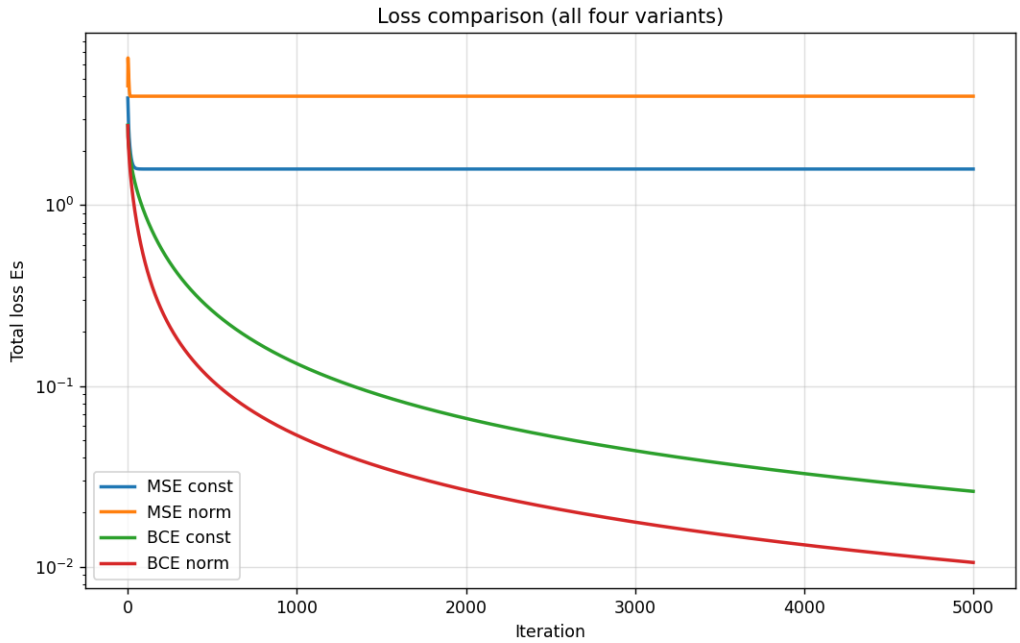
```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['toolbar'] = 'None'

points = np.array([
    [ 2.0,  6.0],
    [-2.0,  6.0],
    [ 2.0, -6.0],
    [-2.0, -6.0]
], dtype=float)

labels = np.array([0, 1, 0, 0], dtype=float)
targets_mse = np.where(labels == 0, -1.0, 1.0)

XMIN, XMAX = -3.0, 3.0
YMIN, YMAX = -7.0, 7.0

tol = 0.01
lr_const = 0.01
MAX_ITER = 5000


def hardlim(value):
    return 1 if value >= 0 else 0


def logistic(u):
    u = np.clip(u, -500, 500)
    return 1.0 / (1.0 + np.exp(-u))


def compute_output(vec, weights, bias):
    return float(np.dot(weights, vec) + bias)


def classify_mse(points, w, bias):
    return np.array([hardlim(compute_output(p, w, bias)) for p in points], dtype=int)


def classify_bce_proba(points, w, bias):
    return np.array([logistic(compute_output(p, w, bias)) for p in points], dtype=float)


def classify_bce(points, w, bias):
    probs = classify_bce_proba(points, w, bias)
    return np.array([1 if pr >= 0.5 else 0 for pr in probs], dtype=int)


def binary_cross_entropy(true_val, pred_val):
    pred_val = np.clip(pred_val, 1e-12, 1 - 1e-12)
    return -(true_val * np.log(pred_val) + (1 - true_val) * np.log(1 - pred_val))
```

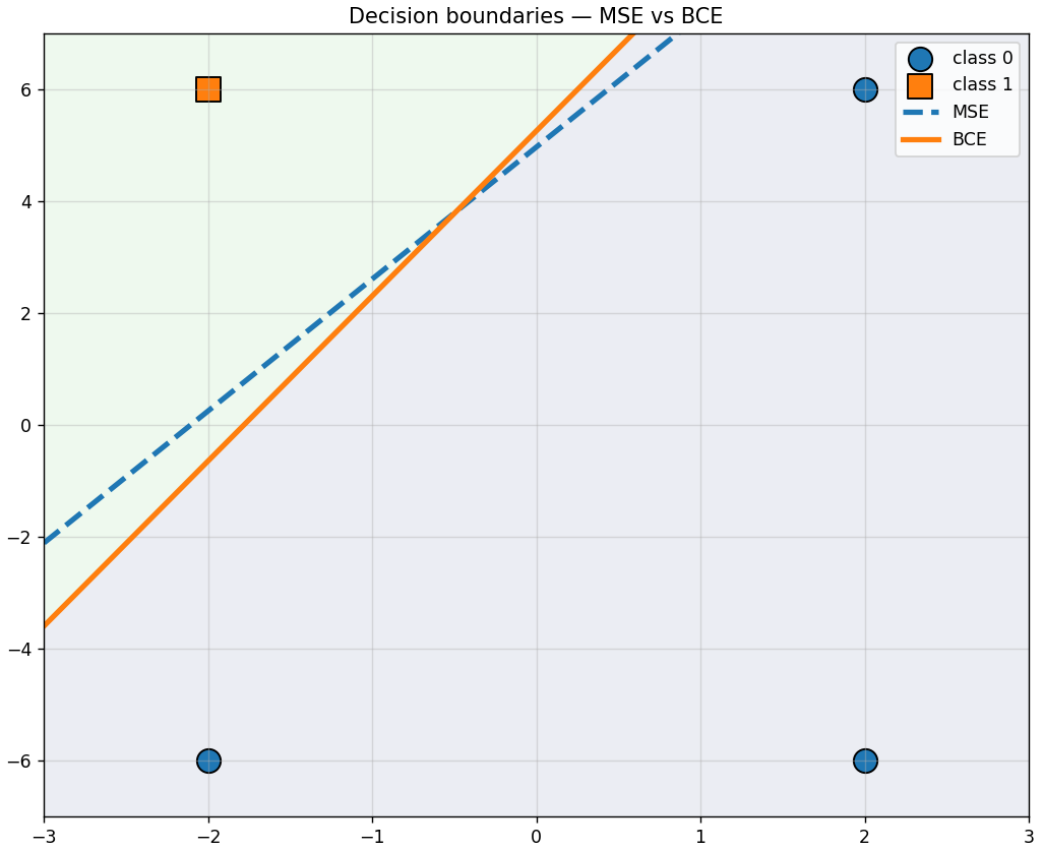
```
def train_mse_constant_step(samples, targets, learning_rate=0.01, tolerance=0.01, maxiter=5000):
    weights = np.zeros(samples.shape[1], dtype=float)
    bias = 0.0
    loss_history = []
    iteration = 0

    while iteration < maxiter:
        total_loss = 0.0
        for vec, tgt in zip(samples, targets):
            pred = compute_output(vec, weights, bias)
            delta = tgt - pred
            total_loss += delta * delta
            weights += learning_rate * delta * vec
            bias    += learning_rate * delta
        loss_history.append(total_loss)
        iteration += 1
        if total_loss <= tolerance:
            break

    return weights, bias, np.array(loss_history), iteration


def train_mse_normalized(samples, targets, tolerance=0.01, maxiter=5000):
    weights = np.zeros(samples.shape[1], dtype=float)
    bias = 0.0
    loss_history = []
    iteration = 0

    while iteration < maxiter:
        total_loss = 0.0
        for vec, tgt in zip(samples, targets):
            pred = compute_output(vec, weights, bias)
            delta = tgt - pred
            total_loss += delta * delta

            denom = np.dot(vec, vec) + 1.0
            step_size = 1.0 / denom if denom != 0 else 0.0

            weights += step_size * delta * vec
            bias    += step_size * delta

        loss_history.append(total_loss)
        iteration += 1
        if total_loss <= tolerance:
            break

    return weights, bias, np.array(loss_history), iteration


def train_bce_constant_step(samples, targets, learning_rate=0.01, tolerance=0.01, maxiter=5000):
    weights = np.zeros(samples.shape[1], dtype=float)
    bias = 0.0
    loss_history = []
    iteration = 0

    while iteration < maxiter:
        total_loss = 0.0
        for vec, tgt in zip(samples, targets):
            z = compute_output(vec, weights, bias)
            prob = logistic(z)
            delta = prob - tgt
            total_loss += binary_cross_entropy(tgt, prob)

            weights -= learning_rate * delta * vec
            bias    -= learning_rate * delta

        loss_history.append(total_loss)
        iteration += 1
        if total_loss <= tolerance:
            break

    return weights, bias, np.array(loss_history), iteration


def train_bce_normalized(samples, targets, tolerance=0.01, maxiter=5000):
    weights = np.zeros(samples.shape[1], dtype=float)
    bias = 0.0
    loss_history = []
    iteration = 0

    while iteration < maxiter:
        total_loss = 0.0
        for vec, tgt in zip(samples, targets):
            z = compute_output(vec, weights, bias)
            prob = logistic(z)
            delta = prob - tgt
            total_loss += binary_cross_entropy(tgt, prob)

            denom = np.dot(vec, vec) + 1.0
            step_size = 1.0 / denom if denom != 0 else 0.0

            weights -= step_size * delta * vec
```

```
            bias    -= step_size * delta

        loss_history.append(total_loss)
        iteration += 1
        if total_loss <= tolerance:
            break

    return weights, bias, np.array(loss_history), iteration


def visualize_loss_curves(loss_mse_const, loss_mse_norm, loss_bce_const, loss_bce_norm):
    plt.figure(figsize=(10, 6))

    for arr in (loss_mse_const, loss_mse_norm, loss_bce_const, loss_bce_norm):
        arr[:] = np.clip(arr, 1e-12, 1e6)

    plt.plot(loss_mse_const, lw=2, label="MSE const")
    plt.plot(loss_mse_norm,  lw=2, label="MSE norm")
    plt.plot(loss_bce_const, lw=2, label="BCE const")
    plt.plot(loss_bce_norm,  lw=2, label="BCE norm")

    plt.title("Loss comparison (all four variants)")
    plt.xlabel("Iteration")
    plt.ylabel("Total loss Es")
    plt.yscale("log")
    plt.grid(True, alpha=0.4)
    plt.legend()
    plt.show()


def draw_decision_boundaries(points, labels, w_mse, b_mse, w_bce, b_bce, extra_point=None):
    fig = plt.figure(figsize=(10, 8))
    ax = fig.gca()

    ax.set_xlabel("x₁")
    ax.set_ylabel("x₂")
    ax.set_title("Decision boundaries — MSE vs BCE")

    xx, yy = np.meshgrid(
        np.linspace(XMIN, XMAX, 300),
        np.linspace(YMIN, YMAX, 300)
    )
    Z = w_bce[0] * xx + w_bce[1] * yy + b_bce
    probs = logistic(Z)
    ax.contourf(xx, yy, probs, levels=[0, 0.5, 1], alpha=0.10)

    cls0 = points[labels == 0]
    cls1 = points[labels == 1]

    if len(cls0): ax.scatter(cls0[:,0], cls0[:,1], s=180, marker="o", edgecolor="k", lw=1.1, label="class 0")
    if len(cls1): ax.scatter(cls1[:,0], cls1[:,1], s=180, marker="s", edgecolor="k", lw=1.1, label="class 1")

    xline = np.linspace(XMIN, XMAX, 300)

    if abs(w_mse[1]) > 1e-11:
        yline = -(w_mse[0] * xline + b_mse) / w_mse[1]
        ax.plot(xline, yline, lw=3, ls="--", label="MSE")
    elif abs(w_mse[0]) > 1e-11:
        ax.axvline(-b_mse / w_mse[0], lw=3, ls="--", label="MSE")

    if abs(w_bce[1]) > 1e-11:
        yline = -(w_bce[0] * xline + b_bce) / w_bce[1]
        ax.plot(xline, yline, lw=3, label="BCE")
    elif abs(w_bce[0]) > 1e-11:
        ax.axvline(-b_bce / w_bce[0], lw=3, label="BCE")

    if extra_point is not None:
        pt = np.asarray(extra_point, dtype=float)
        pr = logistic(compute_output(pt, w_bce, b_bce))
        cl = 1 if pr >= 0.5 else 0
        mk = "*" if cl == 1 else "x"
        ax.scatter([pt[0]], [pt[1]], marker=mk, s=240, lw=2.3,
                   label=f"test point: cls={cl}, p={pr:.3f}")

    ax.set_xlim(XMIN, XMAX)
    ax.set_ylim(YMIN, YMAX)
    ax.grid(True, alpha=0.4)
    ax.legend()
    plt.show()


def interactive_classification_demo(w_bce, b_bce, w_mse, b_mse):
    plt.ion()
    fig, ax = plt.subplots(figsize=(10, 8))

    ax.set_xlim(XMIN, XMAX)
    ax.set_ylim(YMIN, YMAX)
    ax.set_xlabel("x₁")
    ax.set_ylabel("x₂")
    ax.set_title("Interactive classification (BCE normalized)")
```

```
    xx, yy = np.meshgrid(np.linspace(XMIN,XMAX,300), np.linspace(YMIN,YMAX,300))
    Z = w_bce[0]*xx + w_bce[1]*yy + b_bce
    ax.contourf(xx, yy, logistic(Z), levels=[0,0.5,1], alpha=0.10)

    cls0 = points[labels==0]
    cls1 = points[labels==1]
    if len(cls0): ax.scatter(cls0[:,0], cls0[:,1], s=180, marker="o", edgecolor="k", lw=1.1, label="class 0")
    if len(cls1): ax.scatter(cls1[:,0], cls1[:,1], s=180, marker="s", edgecolor="k", lw=1.1, label="class 1")

    xline = np.linspace(XMIN, XMAX, 300)

    if abs(w_mse[1]) > 1e-11:
        ax.plot(xline, -(w_mse[0]*xline + b_mse)/w_mse[1], lw=3, ls="--", label="MSE")
    elif abs(w_mse[0]) > 1e-11:
        ax.axvline(-b_mse/w_mse[0], lw=3, ls="--", label="MSE")

    if abs(w_bce[1]) > 1e-11:
        ax.plot(xline, -(w_bce[0]*xline + b_bce)/w_bce[1], lw=3, label="BCE")
    elif abs(w_bce[0]) > 1e-11:
        ax.axvline(-b_bce/w_bce[0], lw=3, label="BCE")

    ax.grid(True, alpha=0.4)
    ax.legend()
    fig.canvas.draw()
    fig.canvas.flush_events()

    while True:
        inp = input("→ x1 x2    (или q для выхода): ").strip()
        if inp.lower() in ("q", "quit", "exit", ""):
            break
        try:
            xs = [float(v) for v in inp.replace(",", " ").split() if v.strip()]
            if len(xs) != 2:
                print("Нужно ввести ровно два числа")
                continue
            x1, x2 = xs
        except:
            print("Ошибка ввода")
            continue

        pt = np.array([x1, x2])
        prob = logistic(compute_output(pt, w_bce, b_bce))
        cl = 1 if prob >= 0.5 else 0
        mk = "*" if cl == 1 else "x"

        ax.scatter([x1], [x2], marker=mk, s=240, lw=2.3)
        fig.canvas.draw()
        fig.canvas.flush_events()

        print(f"  p = {prob:.6f}   →   класс {cl}")

    plt.ioff()
    plt.show()


def run_experiment():
    w_mse_c,  b_mse_c,  hist_mse_c,  it_mse_c  = train_mse_constant_step(points, targets_mse, lr_const, tol, MAX_ITER)
    w_mse_n,  b_mse_n,  hist_mse_n,  it_mse_n  = train_mse_normalized(points, targets_mse, tol, MAX_ITER)
    w_bce_c,  b_bce_c,  hist_bce_c,  it_bce_c  = train_bce_constant_step(points, labels, lr_const, tol, MAX_ITER)
    w_bce_n,  b_bce_n,  hist_bce_n,  it_bce_n  = train_bce_normalized(points, labels, tol, MAX_ITER)

    acc_mse_c = np.mean(classify_mse(points, w_mse_c, b_mse_c) == labels.astype(int))
    acc_mse_n = np.mean(classify_mse(points, w_mse_n, b_mse_n) == labels.astype(int))
    acc_bce_c = np.mean(classify_bce(points, w_bce_c, b_bce_c) == labels.astype(int))
    acc_bce_n = np.mean(classify_bce(points, w_bce_n, b_bce_n) == labels.astype(int))

    print("Результаты:")
    print(f"MSE const     →  iter={it_mse_c:4d}   loss={hist_mse_c[-1]:.5e}   acc={acc_mse_c:.2f}")
    print(f"MSE norm      →  iter={it_mse_n:4d}   loss={hist_mse_n[-1]:.5e}   acc={acc_mse_n:.2f}")
    print(f"BCE const     →  iter={it_bce_c:4d}   loss={hist_bce_c[-1]:.5e}   acc={acc_bce_c:.2f}")
    print(f"BCE norm      →  iter={it_bce_n:4d}   loss={hist_bce_n[-1]:.5e}   acc={acc_bce_n:.2f}\n")

    visualize_loss_curves(hist_mse_c, hist_mse_n, hist_bce_c, hist_bce_n)
    draw_decision_boundaries(points, labels, w_mse_c, b_mse_c, w_bce_n, b_bce_n)
    interactive_classification_demo(w_bce_n, b_bce_n, w_mse_c, b_mse_c)


if __name__ == "__main__":
    run_experiment()
```

Результат :





В ходе работы была реализована бинарная классификация с использованием логистической регрессии и функции потерь BCE. Для сравнения рассматривались четыре варианта обучения: MSE с фиксированным шагом, MSE с адаптивным шагом, BCE с фиксированным шагом и BCE с адаптивным шагом. Для каждого метода были построены кривые сходимости ошибки и проанализировано число эпох, необходимых для достижения заданной точности.
Эксперименты показали, что BCE обеспечивает более устойчивое и быстрое обучение по сравнению с MSE, поскольку оптимизирует вероятность принадлежности к классу и лучше соответствует задачам классификации. Использование адаптивного шага дополнительно ускоряет сходимость, автоматически подстраивая величину обновления под текущий входной вектор.

Сравнение разделяющих линий показало, что все методы дают схожие по направлению границы, однако BCE формирует более корректную вероятностную поверхность, что улучшает интерпретацию результата.
Таким образом, для бинарной классификации предпочтительно использовать BCE в сочетании с сигмоидой, а применение адаптивного шага делает обучение быстрее и стабильнее.

Вывод:

Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провел сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2)